

§ 9.2 *跳转表

第2章所介绍的有序向量和第3章所介绍的有序列表，各有所长：前者便于静态查找，但动态维护成本较高；后者便于增量式的动态维护，但只能支持顺序查找。为结合二者的优点，同时弥补其不足，第7章和第8章逐步引入了平衡二叉搜索树，其查找、插入和删除操作均可在 $O(\log n)$ 时间内完成。尽管如此，这些结构的相关算法往往较为复杂，代码实现和调试的难度较大，其正确性、鲁棒性和可维护性也很难保证。

设计并引入跳转表^②（skip list）结构的初衷，正是在于试图找到另外一种简便直观的方式，来完成这一任务。具体地，跳转表是一种高效的词典结构，它的定义与实现完全基于第3章的有序列表结构，其查询和维护操作在平均的意义下均仅需 $O(\log n)$ 时间。

9.2.1 Skiplist模板类

跳转表结构以模板类形式定义的接口，如代码9.2所示。

```

1 #include "../List/List.h" //引入列表
2 #include "../Entry/Entry.h" //引入词条
3 #include "Quadlist.h" //引入Quadlist
4 #include "../Dictionary/Dictionary.h" //引入词典
5
6 template <typename K, typename V> //key、value
7 //符合Dictionary接口的Skiplist模板类（但隐含假设元素之间可比较大小）
8 class Skiplist : public Dictionary<K, V>, public List<Quadlist<Entry<K, V>>*> {
9 protected:
10     bool skipSearch(
11         ListNode<Quadlist<Entry<K, V>>*> &qlist,
12         QuadlistNode<Entry<K, V>>* &p,
13         K& k);
14 public:
15     int size() const { return empty() ? 0 : last()->data->size(); } //底层Quadlist的规模
16     int level() { return List::size(); } //层高 == #Quadlist，不一定要开放
17     bool put(K, V); //插入（注意与Map有别——Skiplist允许词条重复，故必然成功）
18     V* get(K k); //读取
19     bool remove(K k); //删除
20 };

```

代码9.2 Skiplist模板类

可见，借助多重继承（multiple inheritance）机制，由Dictionary和List共同派生而得的Skiplist模板类，同时具有这两种结构的特性；此外，这里还重写了在Dictionary抽象类（代码9.1）中，以虚函数形式定义的get()、put()和remove()等接口。

^② 由W. Pugh于1989年发明^[52]

9.2.2 总体逻辑结构

跳转表的宏观逻辑结构如图9.2所示。其内部由沿横向分层、沿纵向相互耦合的多个列表 $\{S_0, S_1, S_2, \dots, S_h\}$ 组成， h 称作跳转表的高度。

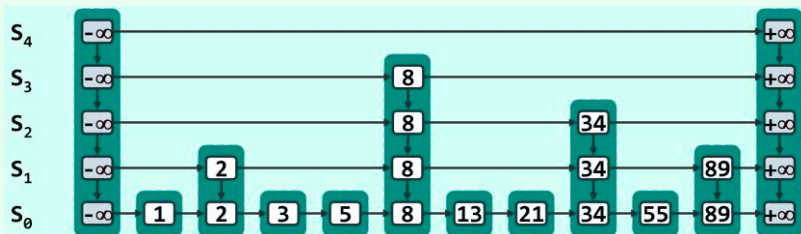


图9.2 跳转表的总体逻辑结构

每一水平列表称作一层（level），其中 S_0 和 S_h 分别称作底层（bottom）和顶层（top）。与通常的列表一样，同层节点之间可定义前驱与后继关系。为便于查找，同层节点都按关键码排序。需再次强调的是，这里的次序只是内部的一种约定；对外部而言，各词条之间仍然只需支持判等操作即可。为简化算法实现，每层列表都设有头、尾哨兵节点。

层次不同的节点可能沿纵向组成塔（tower），同一塔内的节点以高度为序也可定义前驱与后继关系。塔与词典中的词条一一对应。尽管塔内的节点相互重复，但正如随后将要看到的，这种重复不仅可以加速查找，而且只要策略得当，也不至造成空间的实质浪费。

高层列表总是低层列表的子集，其中特别地， S_0 包含词典中的所有词条，而 S_h 除头、尾哨兵外不含任何实质的词条。不难看出，跳转表的层高 h 必然决定于最大的塔高。

9.2.3 四联表

按上述约定，跳转表内各节点沿水平和垂直方向都可定义前驱和后继，支持这种联接方式的表称作四联表（quadlist），它也是代码9.2中Skiplist模板类的底层实现方式。

■ Quadlist模板类

四联表结构可如代码9.3所示，以模板类的形式定义接口。

```

1 #include "QuadlistNode.h" //引入Quadlist节点类
2 template <typename T> class Quadlist { //Quadlist模板类
3 private:
4     int _size; QuadlistNodePosi(T) header; QuadlistNodePosi(T) trailer; //规模、头哨兵、尾哨兵
5 protected:
6     void init(); //Quadlist创建时的初始化
7     int clear(); //清除所有节点
8 public:
9     // 构造函数
10    Quadlist() { init(); } //默认
11    // 析构函数
12    ~Quadlist() { clear(); delete header; delete trailer; } //删除所有节点，释放哨兵
13    // 只读访问接口

```

```

14 Rank size() const { return _size; } //规模
15 bool empty() const { return _size <= 0; } //判空
16 QuadlistNodePosi(T) first() const { return header->succ; } //首节点位置
17 QuadlistNodePosi(T) last() const { return trailer->pred; } //末节点位置
18 bool valid(QuadlistNodePosi(T) p) //判断位置p是否对外合法
19 { return p && (trailer != p) && (header != p); }
20 // 可写访问接口
21 T remove(QuadlistNodePosi(T) p); //删除（合法）位置p处的节点，返回被删除节点的数值
22 QuadlistNodePosi(T) //将*e作为p的后继、b的上邻插入
23 insertAfterAbove(T const& e, QuadlistNodePosi(T) p, QuadlistNodePosi(T) b = NULL);
24 // 遍历
25 void traverse(void (*)(T&)); //遍历各节点，依次实施指定操作（函数指针，只读或局部修改）
26 template <typename VST> //操作器
27 void traverse(VST&); //遍历各节点，依次实施指定操作（函数对象，可全局性修改节点）
28 }; //Quadlist

```

代码9.3 Quadlist模板类

此处定义的接口包括：定位首节点、末节点，在全表或某一区间查找具有特定关键码的节点，删除特定节点，以及插入特定节点。通过它们的相互组合，即可实现跳转表相应的接口功能。

■ 四联表节点

作为四联表的基本组成元素，节点QuadlistNode模板类可定义如代码9.4所示。

```

1 #include "../Entry/Entry.h"
2 #define QuadlistNodePosi(T) QuadlistNode<T>* //跳转表节点位置
3
4 template <typename T> struct QuadlistNode { //QuadlistNode模板类
5     T entry; //所存词条
6     QuadlistNodePosi(T) pred; QuadlistNodePosi(T) succ; //前驱、后继
7     QuadlistNodePosi(T) above; QuadlistNodePosi(T) below; //上邻、下邻
8     QuadlistNode //构造器
9     ( T e = T(), QuadlistNodePosi(T) p = NULL, QuadlistNodePosi(T) s = NULL,
10     QuadlistNodePosi(T) a = NULL, QuadlistNodePosi(T) b = NULL)
11     : entry(e), pred(p), succ(s), above(a), below(b) {}
12     QuadlistNodePosi(T) insertAsSuccAbove //插入新节点，以当前节点为前驱，以节点b为下邻
13     (T const& e, QuadlistNodePosi(T) b = NULL);
14 };

```

代码9.4 QuadlistNode模板类

为简化起见，这里并未做严格封装。对应于水平的前驱与后继，这里为每个节点设置了一对指针pred和succ；垂直方向的上邻和下邻则对应于above和below。主要的操作接口只有insertAsSuccAbove()，它负责创建新节点，并将其插入于当前节点之后、节点b之上。

■ 初始化与构造

由代码9.3可见，四联表的构造，实际上是通过调用如下init()函数完成的。

```
1 template <typename T> void Quadlist<T>::init() { //Quadlist初始化, 创建Quadlist对象时统一调用
2     header = new QuadlistNode<T>; //创建头哨兵节点
3     trailer = new QuadlistNode<T>; //创建尾哨兵节点
4     header->succ = trailer; header->pred = NULL; //沿横向联接哨兵
5     trailer->pred = header; trailer->succ = NULL; //沿横向联接哨兵
6     header->above = trailer->above = NULL; //纵向的后继置空
7     header->below = trailer->below = NULL; //纵向的前驱置空
8     _size = 0; //记录规模
9 } //如此构造的四联表, 不含任何实质的节点, 且暂时与其它四联表相互独立
```

代码9.5 Quadlist对象的创建

9.2.4 查找

查找是跳转表至关重要和最实质的操作，词条的插入和删除等其它操作均以之为基础，其实现效率也将直接影响到跳转表结构的整体性能。

■ get()

在跳转表中查找关键码k的具体过程，如代码9.6所示。

```
1 template <typename K, typename V> V* Skiplist<K, V>::get(K k) { //跳转表词条查找算法
2     if (empty()) return NULL;
3     ListNode<Quadlist<Entry<K, V>>>* qlist = first(); //从顶层Quadlist的
4     QuadlistNode<Entry<K, V>>* p = qlist->data->first(); //首节点开始
5     return skipSearch(qlist, p, k) ? &(p->entry.value) : NULL; //查找并报告
6 } //有多个命中时靠后者优先
```

代码9.6 Skiplist::get()查找

■ skipSearch()

由上可见，实质的查找过程，只不过是从小层列表qlist的首节点first()出发，调用如代码9.7所示的内部函数skipSearch()。

```
1 /*****
2  * Skiplist词条查找算法 (供内部调用)
3  * 入口: qlist为顶层列表, p为qlist的首节点
4  * 出口: 若成功, p为命中关键码所属塔的顶部节点, qlist为p所属列表
5  *       否则, p为所属塔的基座, 该塔对应于不大于k的最大且最靠右关键码, qlist为空
6  * 约定: 多个词条命中时, 沿四联表取最靠后者
7  *****/
8 template <typename K, typename V> bool Skiplist<K, V>::skipSearch(
9     ListNode<Quadlist<Entry<K, V>>>* &qlist, //从指定层qlist的
10     QuadlistNode<Entry<K, V>>* &p, //首节点p出发
11     K& k) { //向右、向下查找目标关键码k
```

```

12  while (true) { //在每一层
13      while (p->succ && (p->entry.key <= k)) //从前向后查找
14          p = p->succ; //直到出现更大的key或溢出至trailer
15      p = p->pred; //此时倒回一步，即可判断是否
16      if (p->pred && (k == p->entry.key)) return true; //命中
17      qlist = qlist->succ; //否则转入下一层
18      if (!qlist->succ) return false; //若已到穿透底层，则意味着失败
19      p = (p->pred) ? p->below : qlist->data->first(); //否则转至当前塔的下一节点
20  } //课后：通过实验统计，验证关于平均查找长度的结论
21 }

```

代码9.7 Skiplist::skipSearch()查找

这里利用参数 p 和 $qlist$ ，分别指示命中关键码所属塔的顶部节点，及其所属的列表。 $qlist$ 和 p 的初始值分别为顶层列表及其首节点，返回后它们将为上层的查找操作提供必要的信息。

■ 实例

仍以图9.2为例，针对关键码21的查找经过节点 $\{-\infty, -\infty, 8, 8, 8, 8, 13\}$ ，最终抵达21后报告成功；针对关键码34的查找经过节点 $\{-\infty, -\infty, 8, 8\}$ ，最终抵达34后报告成功；针对关键码1的查找经过节点 $\{-\infty, -\infty, -\infty, -\infty, -\infty\}$ ，最终抵达1后报告成功。而针对关键码80的查找经过节点 $\{-\infty, -\infty, 8, 8, 34, 34, 34, 55\}$ ，最终抵达89后报告失败；针对关键码0的查找经过节点 $\{-\infty, -\infty, -\infty, -\infty, -\infty\}$ ，最终抵达1后报告失败；针对关键码99的查找经过节点 $\{-\infty, -\infty, 8, 8, 34, 34, 89\}$ ，最终抵达 $+\infty$ 后报告失败。

9.2.5 空间复杂度

■ “生长概率逐层减半”条件

不难理解，其中各塔高度的随机分布规律（如最大值、平均值等），对跳转表的总体性能至关重要。反之，若不就此作出显式的限定，则跳转表的时间和空间效率都难以保证。

比如，若将最大塔高（亦即跳转表的层高）记作 h ，则在极端情况下，每个词条所对应塔的高度均有可能接近甚至达到 h 。果真如此，在查找及更新过程中需要访问的节点数量将难以控制，时间效率注定会十分低下。同时，若词条总数为 n ，则在此类情况下，跳转表所需的存储空间量也将高达 $\Omega(nh)$ 。

然而幸运的是，若能采用简明而精妙的策略，控制跳转表的生长过程，则在时间和空间方面都可实现足够高的效率。就效果而言，此类控制策略必须满足所谓“生长概率逐层减半”条件：

对于任意 $0 \leq k < h$ ， S_k 中任一节点在 S_{k+1} 中依然出现的概率，始终为 $1/2$

也就是说， S_0 中任一关键码依然在 S_k 中出现的概率，等于 2^{-k} 。这也可等效地理解和模拟为，在各塔自底而上逐层生长的过程中，通过投掷正反面等概率的理想硬币（fair coin），来决定是否继续增长一层——亦即，对应于当前的词条，是否在上一层列表中再插入一个节点。

那么，在插入词条的过程中，应该如何从技术上保证这一条件始终成立呢？具体的方法稍后将在9.2.7节介绍，目前不妨暂且假定这一条件的确成立。

■ 节点总数的期望值

根据数学归纳法,“生长概率逐层减半”条件同时也意味着,列表 S_0 中任一节点在列表 S_k 中依然出现的概率均为 $1/2^k = 2^{-k}$ 。因此,第 k 层列表所含节点的期望数目为:

$$E(|S_k|) = n \times 2^{-k}$$

亦即,各层列表的规模将随高度上升以50%的比率迅速缩小,故空间总体消耗量的期望值应为:

$$E(\sum_k |S_k|) = \sum_k E(|S_k|) = n \times (\sum_k 2^{-k}) < 2n = O(n)$$

9.2.6 时间复杂度

在由多层四联表组成的跳转表中进行查找,需访问的节点数目是否会实质性地增加?由以上代码9.7中查找算法skipSearch()可见,单次纵向或横向跳转本身只需常数时间,故查找所需的时间应取决于横向、纵向跳转的总次数。那么,是否会因层次过多而导致横向或纵向的跳转过多呢?以下从概率的角度,分别对其平均性能做出估计,并说明其期望值均不超过 $O(\log n)$ 。

■ 期望高度与纵向跳转次数

考查第 k 层列表 S_k 。

S_k 非空,当且仅当 S_0 所含的 n 个节点中,至少有一个会出现在 S_k 中,相应的概率应为:

$$\Pr(|S_k| > 0) \leq n \times 2^{-k} = n/2^k$$

反过来, S_k 为空的概率即为:

$$\Pr(|S_k| = 0) \geq 1 - n/2^k$$

可以看出,这一概率将随着高度 k 的增加,而迅速上升并接近100%。

以第 $k = 3 \cdot \log n$ 层为例。该层列表 S_k 为空,当且仅当 $h < k$,对应的概率为:

$$\Pr(h < k) = \Pr(|S_k| = 0) \geq 1 - n/2^k = 1 - n/n^3 = 1 - 1/n^2$$

一般地, $k = a \cdot \log n$ 层列表为空的概率为 $1 - 1/n^{a-1}$, $a > 3$ 后这一概率将迅速地接近100%。这意味着跳转表的高度 h 有极大的可能不会超过 $3 \cdot \log n$, h 的期望值应为:

$$E(h) = O(\log n)$$

按照代码9.7的skipSearch()算法,查找过程中的跳转只能向右或向下(而不能向左倒退或向上爬升),故活跃节点的高度必单调非增,每一高度上的纵向跳转至多一次。因此,整个查找过程中消耗于纵向跳转的期望时间不超过跳转表高度 h 的期望值 $O(\log n)$ 。

■ 横向跳转

skipSearch()算法中的内循环对应于沿同一列表的横向跳转,且此类跳转在同一高度可做多次。那么,横向跳转与上述纵向跳转的这一差异,是否意味着这方面的时间消耗将不受跳转表高度 h 的控制,并进而对整体的查找时间产生实质性影响?答案是否定的。

进一步观察skipSearch()算法可知,沿同一列表的横向跳转所经过的节点必然依次紧邻,而且它们都应该是各自所属塔的塔顶。若将同层连续横向跳转的次数记作 Y ,则对于任意的 $0 \leq k$, Y 取值为 k 对应于“ k 个塔顶再加最后一个非塔顶”联合事件,故其概率应为:

$$\Pr(Y = k) = (1 - p)^k \cdot p$$

这是一个典型的几何分布(geometric distribution),其中 $p = 1/2$ 是塔继续生长的概率。因此, Y 的期望值应为:

$$E(Y) = (1 - p)/p = (1 - 1/2)/(1/2) = 1$$

也就是说，在同一高度上，彼此紧邻的塔顶节点数目的期望值为 $1 + 1 = 2$ ；沿着每条查找路径，在每一高度上平均只做常数次横向跳转。因此，整个查找过程中所做横向跳转的期望次数，应依然线性正比于跳转表的期望高度，亦即 $O(\log n)$ 。

■ 其它

除以上纵向和横向跳转，`skipSearch()`还涉及其它一些操作，但总量亦不超过 $O(\log n)$ 。比如，内层`while`循环尽管必终止于失败节点（`key`更大或溢出至`trailer`），但此类节点在每层至多一个，访问它们所需的时间总量仍不超过跳转表的期望高度 $E(h) = O(\log n)$ 。

9.2.7 插入

■ `put()`

将词条(`k`, `v`)插入跳转表的具体操作过程，可描述和实现如代码9.8所示。

```
1 template <typename K, typename V> bool Skiplist<K, V>::put(K k, V v) { //跳转表词条插入算法
2     Entry<K, V> e = Entry<K, V>(k, v); //待插入的词条（将被随机地插入多个副本）
3     if (empty()) insertAsFirst(new Quadlist<Entry<K, V>>()); //插入首个Entry
4     ListNode<Quadlist<Entry<K, V>>*> qlist = first(); //从顶层四联表的
5     QuadlistNode<Entry<K, V>>* p = qlist->data->first(); //首节点出发
6     if (skipSearch(qlist, p, k)) //查找适当的插入位置（不大于关键码k的最后一个节点p）
7         while (p->below) p = p->below; //若已有雷同词条，则需强制转到塔底
8     qlist = last(); //以下，紧邻于p的右侧，一座新塔将自底而上逐层生长
9     QuadlistNode<Entry<K, V>>* b = qlist->data->insertAfterAbove(e, p); //新节点b即为新塔基座
10    while (rand() & 1) { //经投掷硬币，若确定新塔需要再长高一层，则
11        while (qlist->data->valid(p) && !p->above) p = p->pred; //找出不低于此高度的最近前驱
12        if (!qlist->data->valid(p)) { //若该前驱是header
13            if (qlist == first()) //且当前已是最顶层，则意味着必须
14                insertAsFirst(new Quadlist<Entry<K, V>>()); //首先创建新的一层，然后
15                p = qlist->pred->data->first()->pred; //将p转至上一层Skiplist的header
16        } else //否则，可径自
17            p = p->above; //将p提升至该高度
18        qlist = qlist->pred; //上升一层，并在该层
19        b = qlist->data->insertAfterAbove(e, p, b); //将新节点插入p之后、b之上
20    } //课后：调整随机参数，观察总体层高的相应变化
21    return true; //Dictionary允许重复元素，故插入必成功——这与Hashtable等Map略有差异
22 }
```

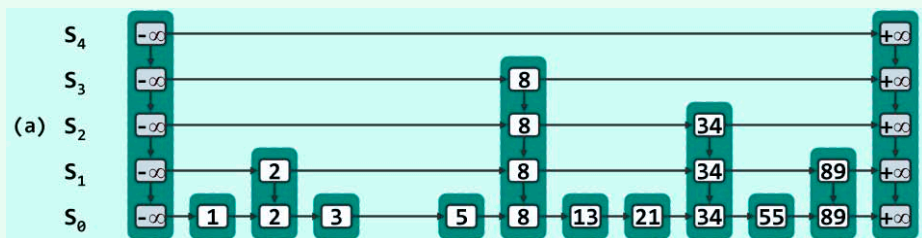
代码9.8 `Skiplist::put()`插入

这里通过逻辑表达式“`rand() % 2`”来模拟投掷硬币，并保证“生长概率逐层减半”条件。也就是说，通过（伪）随机整数的奇偶，近似地模拟一次理想的掷硬币实验。只要（伪）随机数为奇数（等价于掷出硬币正面），新塔就继续生长；一旦取（伪）随机数为偶数（等价于掷出反面），循环随即终止（生长停止），整个插入操作亦告完成。

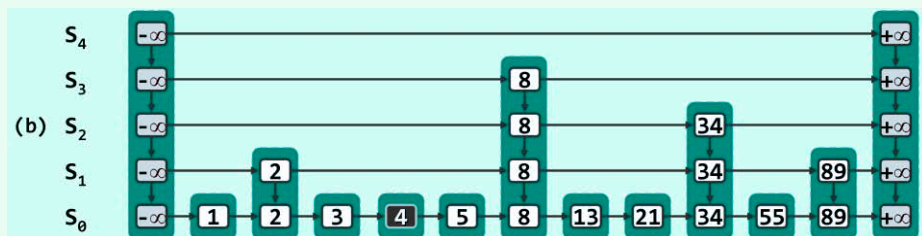
由此可见，新塔最终的（期望）高度，将取决于此前连续的正面硬币事件的（期望）次数。

■ 实例

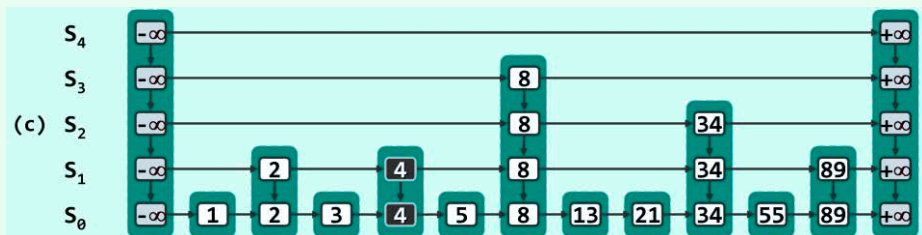
考查如图9.2所示的跳转表。将关键码4插入其中的过程，如图9.3(a~d)所示。



首先如图(a)所示，经过查找确定，应紧邻于关键码3右侧实施插入。



然后如图(b)所示，在底层列表中，创建一个节点作为新塔的基座。



此后，假定随后掷硬币的过程中，前两次为正面，第三次为反面。于是如图(c)和(d)所示，新塔将连续长高两层后停止生长。

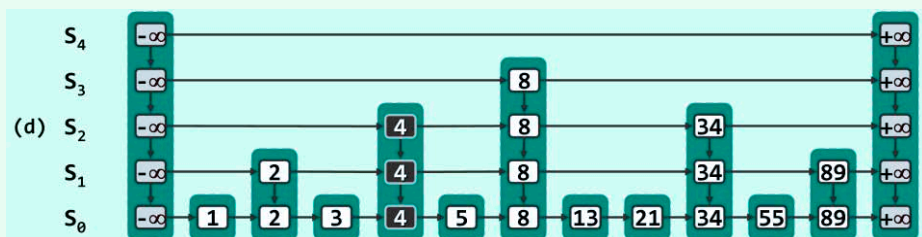


图9.3 跳转表节点插入过程(a~d)，也是节点删除的逆过程(d~a)

新塔每长高一层，塔顶节点除须与原塔纵向联接，还须与所在列表中的前驱与后继横向联接。

■ insertAfterAbove()

可见，QuadlistNode节点总是以塔为单位自底而上地成批插入，且每一节点都是作为当时的新塔顶而插入。也就是说，QuadlistNode节点的插入都属于同一固定的模式：创建关键码为e的新节点，将其作为节点p的后继和节点b（当前塔顶）的上邻“植入”跳转表。

因此，代码9.3只需提供统一的接口insertAfterAbove()，其具体实现如代码9.9所示。


```

1 template <typename T> QuadlistNodePosi(T) //将e作为p的后继、b的上邻插入Quadlist
2 Quadlist<T>::insertAfterAbove(T const& e, QuadlistNodePosi(T) p, QuadlistNodePosi(T) b = NULL)
3 { _size++; return p->insertAsSuccAbove(e, b); } //返回新节点位置 ( below = NULL )

```

代码9.9 Quadlist::insertAfterAbove()插入

■ insertAsSuccAbove()

上述接口的实现，需转而调用节点p的insertAsSuccAbove()接口，如代码9.10所示完成节点插入的一系列实质性操作。

```

1 template <typename T> QuadlistNodePosi(T) //将e作为当前节点的后继、b的上邻插入Quadlist
2 QuadlistNode<T>::insertAsSuccAbove(T const& e, QuadlistNodePosi(T) b = NULL) {
3     QuadlistNodePosi(T) x = new QuadlistNode<T>(e, this, succ, NULL, b); //创建新节点
4     succ->pred = x; succ = x; //设置水平逆向链接
5     if (b) b->above = x; //设置垂直逆向链接
6     return x; //返回新节点的位置
7 }

```

代码9.10 QuadlistNode::insertAsSuccAbove()插入

具体过程如图9.4(a)所示，插入前节点b的上邻总是为空。

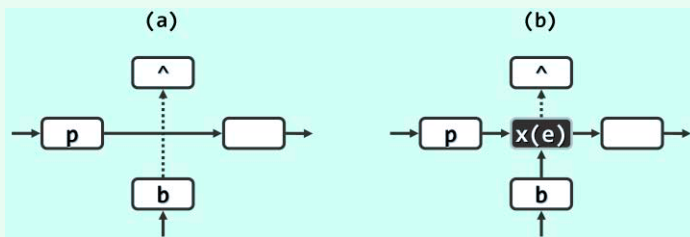


图9.4 四联表节点插入过程

首先，创建一个关键码为e的节点，其前驱和后继分别设为当前节点（p）及其后继（p->succ），上邻和下邻分别设为NULL和节点b。然后，沿水平和垂直方向设置好逆向的链接。最终结果如图(b)所示。

因这里允许关键码雷同，故在插入之前无需查找确认是否已有某个词条的关键码为e。

■ 时间复杂度

新塔每长高一层，都要紧邻于该层的某一节点p之后创建新的塔顶节点。准确地，在不大于新关键码的所有节点中，节点p为最大者；若这样的节点有多个，则按约定，p应取其中最靠后者。然而，若在每一层都从首节点开始，通过扫描确认p的位置，则最坏情况下可能每一层四联表都几乎需要遍历，耗时量将高达 $\Omega(n)$ 。然而实际上，各层四联表中的位置p之间自底而上存在很强的关联性，利用这一性质即可保证高效而精准地确定各高度上的插入位置p。

具体地如代码9.8所示，每次都从当前节点p的前驱出发，先上升一层，然后自右向左依次移动，直到发现新节点在新高度上的前驱。接下来，只需将p更新为该前驱的位置，并将新塔顶节点插入于p之后，新塔顶节点的插入即告完成。实际上，新塔每增长一层，都可重复上述过程完成新塔顶节点的插入。

整个过程中p所经过的路径，与关键码的查找路径恰好方向相反。由9.2.6节的结论，被访问节点的期望总数不超过 $O(\log n)$ ，因此这也是插入算法运行时间期望值的上界。

9.2.8 删除

■ Skiplist::remove()

从跳转表中删除关键码为k词条的具体操作过程，如描述为代码9.11。

```

1 template <typename K, typename V> bool Skiplist<K, V>::remove(K k) { //跳转表词条删除算法
2     if (empty()) return false; //空表情况
3     ListNode<Quadlist<Entry<K, V>>*>* qlist = first(); //从顶层Quadlist的
4     QuadlistNode<Entry<K, V>>* p = qlist->data->first(); //首节点开始
5     if (!skipSearch(qlist, p, k)) return false; //目标词条不存在，直接返回
6     do { //若目标词条存在，则逐层拆除与之对应的塔
7         QuadlistNode<Entry<K, V>>* lower = p->below; //记住下一层节点
8         qlist->data->remove(p); //删除当前层节点
9         p = lower; qlist = qlist->succ; //转入下一层
10    } while (qlist->succ); //直到塔基
11    while (!empty() && first()->data->empty()) //反复
12        List::remove(first()); //清除已可能不含词条的顶层Quadlist
13    return true; //删除操作成功完成
14 }
```

代码9.11 Skiplist::remove()删除

这一过程的次序，与插入恰好相反。以如图9.3(d)所示的跳转表为例，若欲从其中删除关键码为4的词条，则在查找定位该词条后，依次删除塔顶。关键码删除过程的中间结果如图(c)和(b)所示，最终结果如图(a)。

■ Quadlist::remove()

在基于四联表实现跳转表中，QuadlistNode节点总是以塔为单位，自顶而下地成批被删除，其中每一节点的删除，都按照如下固定模式进行：节点p为当前的塔顶，将它从所属横向列表中删除；其下邻（若存在）随后将成为新塔顶，并将在紧随其后的下一次删除操作中被删除。

Quadlist模板类（代码9.3）为此定义了接口remove()，其具体实现如代码9.12所示。

```

1 template <typename T> //删除Quadlist内位置p处的节点，返回其中存放的词条
2 T Quadlist<T>::remove(QuadlistNodePosi(T) p) { //assert: p为Quadlist中的合法位置
3     p->pred->succ = p->succ; p->succ->pred = p->pred; _size--; //摘除节点
4     T e = p->entry; delete p; //备份词条，释放节点
5     return e; //返回词条
6 }
7
8 template <typename T> int Quadlist<T>::clear() { //清空Quadlist
9     int oldSize = _size;
10    while (0 < _size) remove(header->succ); //逐个删除所有节点
11    return oldSize;
12 }
```

代码9.12 Quadlist::remove()删除

这里各步迭代中的操作次序,与图9.4(a)和(b)基本相反。略微不同之处在于,因必然是整塔删除,故可省略纵向链接的调整。

其中`clear()`接口用以删除表中所有节点,在代码9.3中也是析构过程中的主要操作。

■ 时间复杂度

如代码9.11所示,词条删除算法所需的时间,不外乎消耗于两个方面。

首先是查找目标关键码,由9.2.6节的结论可知,这部分时间的期望值不过 $O(\log n)$ 。其次是拆除与目标关键码相对应的塔,这是一个自顶而下逐层迭代的过程,故累计不超过 h 步;另外,由代码9.12可见,各层对应节点的删除仅需常数时间。

综合以上分析可知,跳转表词条删除操作所需的时间不超过 $O(h) = O(\log n)$ 。

§ 9.3 散列表

散列作为一种思想既朴素亦深刻,作为一种技术则虽古老却亦不失生命力,因而在数据结构及算法中占据独特而重要地位。此类方法以最基本的向量作为底层支撑结构,通过适当的散列函数在词条的关键码与向量单元的秩之间建立起映射关系。理论分析和实验统计均表明,只要散列表、散列函数以及冲突排解策略设计得当,散列技术可在期望的常数时间内实现词典的所有接口操作。也就是说,就平均时间复杂度的意义而言,可以使这些操作所需的运行时间与词典的规模基本无关。尤为重要的是,散列技术完全摒弃了“关键码有序”的先决条件,故就实现词典结构而言,散列所特有的通用性和灵活性是其它方式无法比拟的。

以下将围绕散列表、散列函数以及冲突排解三个主题,逐层深入地展开介绍。

9.3.1 完美散列

■ 散列表

散列表(`hashtable`)是散列方法的底层基础,逻辑上由一系列可存放词条(或其引用)的单元组成,故这些单元也称作桶(`bucket`)或桶单元;与之对应地,各桶单元也应按其逻辑次序在物理上连续排列。因此,这种线性的底层结构用向量来实现再自然不过。为简化实现并进一步提高效率,往往直接使用数组,此时的散列表亦称作桶数组(`bucket array`)。若桶数组的容量为 R ,则其中合法秩的区间 $[0, R)$ 也称作地址空间(`address space`)。

■ 散列函数

一组词条在散列表内部的具体分布,取决于所谓的散列(`hashing`)方案——事先在词条与桶地址之间约定的某种映射关系,可描述为从关键码空间到桶数组地址空间的函数:

`hash() : key → hash(key)`

这里的`hash()`称作散列函数(`hash function`)。反过来,`hash(key)`也称作`key`的散列地址(`hashing address`),亦即与关键码`key`相对应的桶在散列表中的秩。

■ 实例

以学籍库为例。若某高校2011级共计4000名学生的学号为2011-0000至2011-3999,则可直接使用一个长度为4000的散列表`A[0~3999]`,并取

`hash(key) = key - 20110000`

从而将学号为`x`的学生学籍词条存放于桶单元`A[hash(x)]`。